QuillAudits

Blockchain Security Guidelines

in partnership with

INDIA

India Blockchain Forum

Prasanna Lohar

Founder and President
India Blockchain Forum

"Blockchain's inherent security is a solid foundation, but fortifying it with the latest security guidelines ensures that it remains an unbreakable vault of trust in the digital age. The latest report with QuillAudits on security guidelines is a way forward compass for Blockchain companies through the evolving landscape of trust."

"Although cyber fortresses founded on blockchain technology are robust, their ultimate strength hinges upon the watchfulness of guardians equipped with stringent and meticulously crafted security protocols."

Col (Dr.) Inderjeet Singh

Co-Founder
India Blockchain Forum

Preetam Rao

Founder and CEO
QuillAudits

"Innovations in Web3 are accelerating and are here to fulfil the diversifying consumer needs. But these solutions are accessible to investors and fraudsters alike.

Through this comprehensive security guidelines report and our tailored security solutions, we are committed to safeguarding emerging Web3 ventures."

Index

1. **Introduction** 03
   - a. Purpose 04
   - b. Scope of Guideline 04
2. **Importance Of Blockchain Security** 05
3. **Security Guidelines** 07
   - a. Token Contracts Security Guidelines 08
   - b. DeFi Security Guidelines 11
   - c. Test Your Smart Contracts Security—Book a Free Consultation Now! 12
   - d. DAO Audit Guidelines 13
   - e. NFT and NFT MarketPlace Audit Guideline 15
   - f. Get Real-Time Web3 Security Alerts! 18
   - g. Wallet Security Audit Guidelines 19
   - h. dApp Security Guidelines 21
4. **Development Process Checklist** 25
   - a. Documentation and Specifications 26
   - b. Solidity Best Practices and Readability 27
   - c. On-Chain vs Off-Chain Computation 28
   - d. Upgradeability 29
   - e. Function Composition 29
   - f. Inheritance 30
   - g. Events 30
   - h. Avoid Known Pitfalls 31
   - i. Newsletter 33
   - j. Gas Optimization 34

Index

k. Testing and Verification 36

l. Web Front-end Security Configuration Requirements 37

m. Server Environment Security Configuration Requirements 38

n. Quill RedTeam 39

5. Incidence Response 40

a. Detection and Monitoring 41

b. Implement Tools to Monitor Smart Contract Activity For 42

c. Implement Bug Bounty and Responsible Disclosure 42

d. Incident Detection & Reporting Training 43

e. Regular Health Checks of Monitoring Systems 43

f. Pause the Smart Contract Immediately to Prevent Further Damage 44

g. Block Unauthorized Transactions from the Hacker's Addresses 44

h. Isolate the Compromised Smart Contract from Other Systems 45

i. Revoke Read/Write Access for Any Compromised Private Keys 45

j. Eradication and Recovery 46

k. QuillAudits WAGSI Grants 47

l. Investigations and Forensics 48

m. Communication 48

n. Conduct a "Postmortem" to Identify 49

01

Introduction

Purpose

The goal of this paper is to present an in-depth guideline for safeguarding blockchain system security. As blockchain technology evolves and becomes more widely adopted, it is critical to provide a standardised framework outlining best practices and safety measures to prevent risks and vulnerabilities connected with blockchain deployments.

This document addresses the emerging security concerns around blockchain networks, dApps, and smart contracts. Organisations and individuals can improve the overall security posture of their blockchain infrastructure by following the guidelines offered here, reducing the risk of unauthorised access, data breaches, fraud, and other malicious activities.

Scope of Guideline

Security standards Various Tokens, Defi smart contracts, NFT Smart contracts, Dao Smart Contracts, Dapp Security, Wallet Security, Security Best practices, Development Best Practises, and Incident Response are included in the scope of the recommendations.

02

Importance Of Blockchain Security

Importance Of Blockchain Security

Security is of utmost importance in blockchain technology and its associated components. It serves as the foundation for trust, reliability, and protecting assets and sensitive information. Robust security measures safeguard against malicious attacks, fraudulent activities, and unauthorized modifications, ensuring the integrity and immutability of data stored on the blockchain. Furthermore, secure smart contracts, decentralized applications (DApps), and wallets provide users with the confidence to interact with the blockchain ecosystem, knowing that their transactions, digital identities, and assets are protected. By prioritizing security, organizations and individuals can mitigate risks, preserve privacy, and foster a secure environment that encourages the broader adoption and utilization of blockchain technology across diverse domains and industries.

03

Security Guidelines

Token Contracts Security Guidelines

Code Quality:

The first step is to review the overall quality of the code. This includes checking for code clarity, proper comments, and adherence to coding standards.

Token Standards Compliance:

Ensure that the token contract complies with the relevant standards, such as ERC20, ERC721, or ERC1155. Non-compliance can lead to interoperability issues with wallets and exchanges.

Overflow and Underflow Attacks:

Check for potential integer overflow and underflow vulnerabilities. These occur when arithmetic operations exceed the maximum or minimum size of the integer type.

Reentrancy Attacks:

Look for potential reentrancy attacks. This happens when a contract calls an external contract that, in turn, calls back into the calling contract before the first call is finished.

Front-Running Attacks:

Check for susceptibility to front-running attacks where someone can see a transaction in the mempool and then issue another transaction with a higher gas price to get it mined first.

Timestamp Dependence:

Ensure that the contract does not rely on block.timestamp or now for critical functionalities, as miners can manipulate them to a certain degree.

DoS with Unexpected Revert:

Check for functions that could be blocked by an unexpected revert in a called contract, leading to a Denial of Service (DoS) attack.

Short Address Attack:
Ensure the contract is not vulnerable to the short address attack, where an attacker sends fewer data than expected, and Solidity fills shorter addresses with zeroes.

Unchecked Return Values:
Ensure all external calls' return values are properly checked. Failing to do so can lead to unexpected behaviour.

Race Conditions:
Check for potential race conditions, where the order of transactions can affect the contract's state.

Gas Limit and Loops:
Ensure that loops do not have a high upper limit, which could consume all the gas and cause transactions to fail.

Access Control:
Review the contract's access control mechanisms. Ensure that only authorised addresses can call sensitive functions.

Upgradability and Proxy Patterns:
If the contract uses upgradability or proxy patterns, ensure the implementation is secure and cannot be exploited.

Randomness:
If the contract uses randomness, ensure that it is generated securely. Blockchain is deterministic, so generating random numbers can be tricky and potentially exploitable.

Data Storage:
Review how data is stored and handled. Ensure that sensitive data is not publicly accessible.

Contract Interactions:

Review all external contract interactions. Ensure that interactions are with trusted contracts and that potential failures are handled correctly.

Event Logging:

Ensure that all significant actions and state changes are logged using events. This helps with transparency and tracking.

Function Visibility:

Check that all functions have the correct visibility. Public and external functions can be called by anyone, so ensure that sensitive functions are marked as private or internal.

Fallback Functions:

Review the contract's fallback function. Ensure it does not consume too much gas and cannot be exploited.

Self-Destruct:

If the contract uses self-destruct, ensure it can only be called by the appropriate parties and under the correct conditions.

DeFi Security Guidelines

Code Quality:

Check if the code is clean, well-structured, and follows best practices. Look for any signs of rushed or sloppy coding.

Functionality:

Ensure that all functions and modules work as intended. Check if the code logic matches the project's documentation and intended functionality.

Security:

Look for potential security issues, such as reentrancy attacks, integer overflow and underflow, and front-running attacks. Check if the project has implemented measures to prevent these attacks.

Smart Contracts:

Review the smart contracts used in the project. Check if they are secure and if they have been properly tested. Look for any potential vulnerabilities in the smart contracts.

Dependencies:

Check the project's dependencies. Ensure that they are up to any known vulnerabilities.

Data Storage:

Review how the project stores data. Check if it uses encryption for sensitive data and follows best data storage practices.

Error Handling:

Check how the project handles errors. Ensure that it does not reveal any sensitive information when an error occurs.

Gas Optimization:

In Ethereum-based projects, check for gas optimisation in the smart contracts. Inefficient contracts can lead to higher costs for users.

Upgradability:

Check if the smart contracts are upgradeable. If they are, ensure the upgrade process is secure and cannot be exploited.

Access Controls:

Review the project's access controls. Ensure that only authorised individuals can perform certain actions.

Tokenomics:

If the project has its own token, review its tokenomics. Check if the token distribution is fair and if the token has any potential security issues.

Interactions with External Systems:

Review how the project interacts with external systems. Check if these interactions are secure and if they could potentially be exploited.

Test Your Smart Contracts Security— Book a Free Consultation Now!

Get a clear picture of where your Web3 project stands against vulnerabilities and loopholes!

Schedule a Call Now

DAO Audit Guidelines

Governance Mechanisms:
DAOs are governed by their members, so it's crucial to audit the voting mechanisms. Ensure that they are, transparent and can't be easily manipulated.

Proposal Systems:
Review the system for making proposals. Check who can make proposals, how they are on, and how they are executed if approved.

Quorum Requirements:
Check the requirements for a quorum. Ensure it's not set too high (which could prevent any decisions from being made) or too low (which could allow a small group to control the DAO).

Access Controls:
DAOs often have multi-signature wallets or other access control mechanisms. Ensure these are secure and can't be easily exploited.

Token Distribution:
If the DAO has a token, check how it's distributed among members. Ensure that the distribution is fair and aligns with the DAO's goals.

Treasury Management:
Review how the DAO's treasury is managed. Check how funds are spent and ensure checks and balances are in place.

Dispute Resolution:
Check how the DAO resolves disputes among members. This is often done through a voting process, so ensure this is fair and transparent.

Upgradeability:
DAOs often need to upgrade their smart contracts. Check how this process is managed and ensure it's secure.

- **Interactions with Other Contracts:**
  DAOs often interact with other smart contracts (e.g., DeFi protocols). Ensure these interactions are secure.

- **Emergency Shutdown:**
  Check if there's a mechanism for an emergency shutdown in case of a serious issue. This should be a last resort and only accessible to trusted members.

- **Transparency:**
  Ensure that all actions taken by the DAO are transparent and can be audited by members.

- **Member Rights and Responsibilities:**
  Review the rights and responsibilities of DAO members. Ensure they are clearly defined and protected.

NFT And NFT MarketPlace Audit Guideline

Code Quality:

- Check for code readability, modularity, and use of comments.
- Ensure the code follows the Solidity Style Guide.
- Check for the use of the latest Solidity version and compiler settings.

Functionality:

- Verify that the smart contract performs all the functions as described in the whitepaper or documentation.
- Check if the NFTs are ERC721 or ERC1155 compliant.
- Ensure the marketplace contract handles, buying, selling, and bidding of NFTs correctly.

Access Controls:

- Check if the contract has proper access controls.
- Only ensure authorised addresses can call sensitive functions.
- Verify the ownership of the contract and if it can be renounced or transferred.

Token Standards:

- Ensure the NFTs follow the ERC721 or ERC1155 standards.
- Check if the metadata of the NFTs are stored correctly and immutably.
- Verify the implementation of optional ERC721 or ERC1155 functions.

Security:

- Check for common vulnerabilities like reentrancy, overflow and underflow, etc.
- Ensure the contract handles exceptions and fails gracefully.
- Verify if the contract is resistant to front-running attacks.

Testing:

- Ensure the contract has been thoroughly tested.
- Check if all functions have been covered in the tests.
- Verify if edge cases have been tested.

Gas Optimization:

- Check if the contract is optimized for gas usage.
- Ensure expensive operations like SSTORE, SLOAD, and external calls are minimized.

Marketplace Specific Checks:

- Verify the contract handles the listing and delisting of NFTs correctly.
- Check if the contract transfers the ownership of NFTs upon sale.
- Ensure the contract handles the funds correctly and securely.

External Calls:

- Check if the contract makes external calls.
- Ensure the contract handles the possibility of the called contract not executing as expected.

Upgradability:

- Check if the contract is upgradable.
- Ensure the contract handles upgrades securely and correctly.

Contract Logic:

- Verify the contract logic for minting, transferring, and burning NFTs.
- Check if the contract handles royalties correctly.

Contract Interactions:

- Check how the contract interacts with other contracts.
- Ensure the contract handles these interactions securely.

Event Logging:

- Verify that the contract logs events correctly.
- Check if all significant actions are logged.

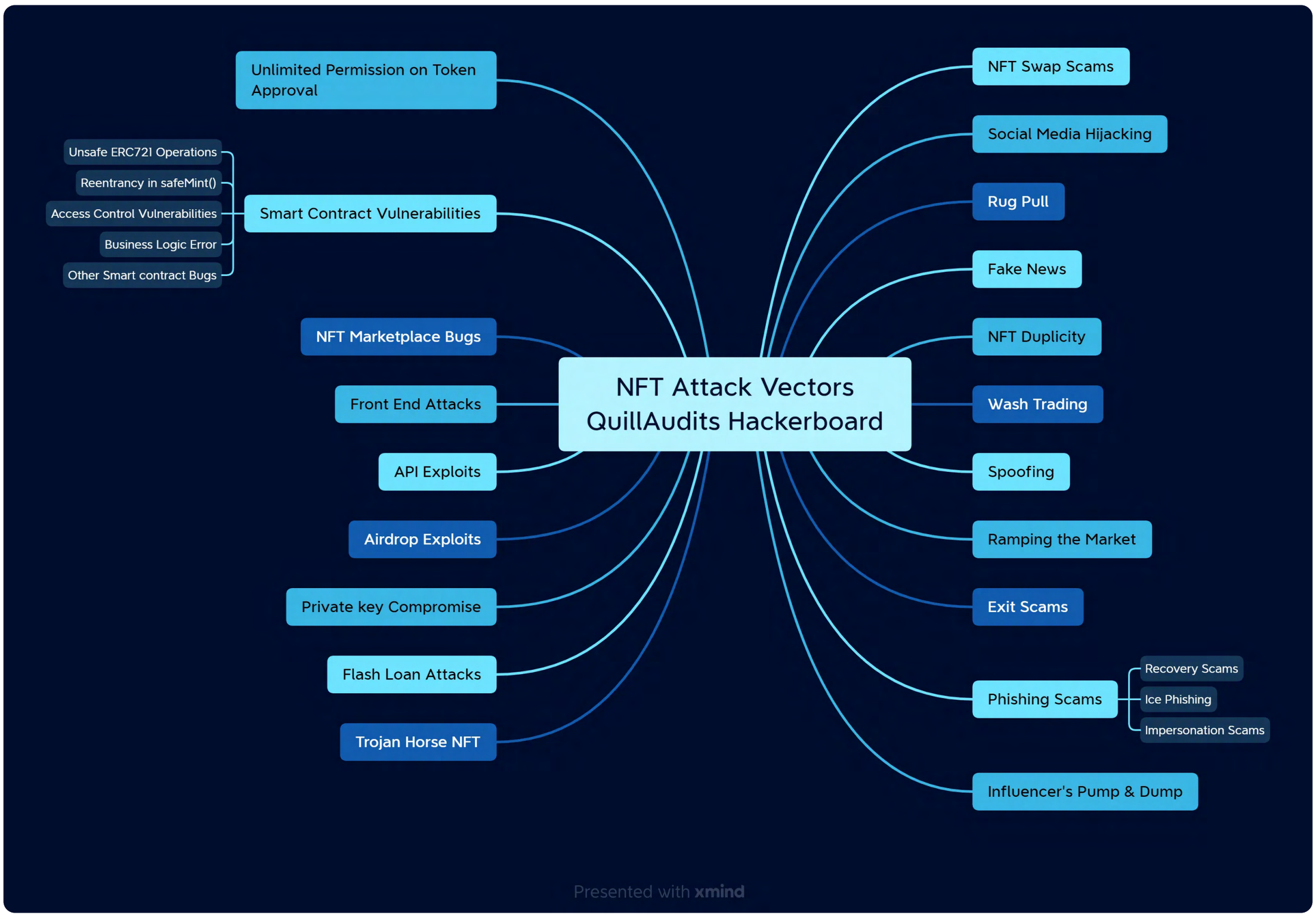
Randomness:

- If the contract uses randomness, ensure it is generated securely.

Contract Termination:

- Check if the contract can be paused or destroyed.
- Ensure these functions can only be called by authorized addresses.

Get Real-Time Web3 Security Alerts!



Access Now!

Wallet Security Audit Guidelines

Wallet Security:

Check if the wallet is secure and encrypted. Test for any vulnerabilities that could allow unauthorized access.

Private Key Security:

Ensure that the private keys are securely stored exposed not and to any third parties.

Transaction Security:

Test the transaction process for any vulnerabilities. This includes checking for any issues with transaction signing, transaction broadcasting, and transaction confirmation.

Authentication Mechanisms:

Check the security of the authentication mechanisms. This includes testing for any vulnerabilities in two-factor authentication (2FA), biometric authentication, and any other authentication methods used.

API If:

Security the wallet uses APIs, these should be tested for any security issues. This includes checking for any vulnerabilities in the API endpoints, the data via transmitted APIs, and the security of the API keys.

Data Encryption:

Test the security of the data encryption methods used by the wallet. This includes checking for any vulnerabilities in the encryption algorithms and the key management systems.

User Privacy:

Check if the wallet respects user privacy. This includes testing for any issues with data collection, data storage, and data sharing.

Recovery Process:
Test the security of the wallet's recovery process. This includes checking for any vulnerabilities in the recovery phase generation and storage, and the recovery process itself.

Server Security:
If the wallet uses a server, this should be tested for any security issues. This includes checking for any vulnerabilities in the server's configuration, the server's software, and the server's network.

DDoS Protection:
Check if the wallet has adequate protection against Distributed Denial of Service (DDoS) attacks.

Phishing Protection:
Test if the wallet has measures in place to protect users from phishing attacks.

Code Review:
Conduct a thorough code review to identify any potential security issues in the wallet's codebase.

Updates and Patches:
Check if the wallet regularly releases updates and patches to fix any known security issues.

Incident Response Plan:
Check if the wallet has an incident response plan in place to handle any security breaches.

Third-Party Integrations:
If the wallet integrates with third-party services, these should be tested for any security issues.

dApp Security Guidelines

Improper Authentication:

Check if the application is using a secure and robust authentication mechanism. Ensure that the application does not allow brute force attacks, dictionary attacks, or any other type of attack that can bypass the authentication process.

Improper Resource Usage:

Verify that the application is using resources efficiently and securely. Check for any resource leaks or improper resource allocation that could lead to performance issues or security vulnerabilities.

Improper Authorization:

Ensure that the application has a proper authorization mechanism in place. Check if the application is correctly enforcing access controls and if it is preventing unauthorized access to sensitive data or functionality.

Insecure File Uploads:

Check if the application allows file uploads and if so, ensure that it is handling them securely. Verify that the application is validating the file type, size, and content before accepting the upload.

Insecure Direct Object References:

Check if the application is vulnerable to insecure direct object references. This occurs when an application provides direct access to objects based on user-supplied input.

Client-Side Validation Issues:

Ensure that the application is not solely relying on client-side validation. Check if the application is also performing server-side validation to prevent bypassing of validation rules.

Rate Limit:
Check if the application has rate limiting in place to prevent brute force attacks and DoS attacks.

Input Validation:
Verify that the application is properly validating all user inputs. Check for any instances of SQL injection, XSS, or other injection attacks.

Injection Attacks:
Check if the application is vulnerable to injection attacks such as SQL injection, OS command injection, or LDAP injection.

Cross-Site Scripting (XSS):
Check if the application is vulnerable to XSS attacks. Verify that the application is properly encoding or escaping user-supplied input.

Cross-Site Request Forgery:
Check if the application is vulnerable to CSRF attacks. Ensure that the application is using anti-CSRF tokens or similar mechanisms to prevent these attacks.

Security Misconfiguration:
Check for any security misconfigurations in the application. This could include insecure default configurations, incomplete or ad hoc configurations, open cloud storage, misconfigured HTTP headers, and verbose error messages containing sensitive information.

Broken Access Controls:
Verify that the application is enforcing access controls correctly. Check for instances where unauthorized users can access sensitive data or functionality.

Insecure Cryptographic Storage:
Check if the application is storing sensitive data securely. Verify that the application is using strong encryption algorithms and that it is managing cryptographic keys securely.

- **Insufficient Cryptography:**
  Check if the application is using weak or outdated cryptographic algorithms. Verify that the application is using strong and up-to-date cryptographic algorithms.

- **Insufficient Session Expiration:**
  Check if the application is properly managing session expiration. Verify that the application is terminating sessions after a period of inactivity and that it is not vulnerable to session fixation attacks.

- **Insufficient Transport Layer Protection:**
  Check if the application is using secure communication protocols such as HTTPS. Verify that the application is not transmitting sensitive data over unencrypted connections.

- **Unvalidated Redirects and Forwards:**
  Check if the application is vulnerable to unvalidated redirects and forwards. This occurs when an application accepts untrusted input that could cause the web application to redirect the request to a URL contained within untrusted input.

- **Information Leakage:**
  Check for any instances where the application is leaking sensitive information. This could include verbose error messages, sensitive data in URLs, or sensitive data in HTTP headers.

- **Broken Authentication and Session Management:**
  Check if the application is properly managing authentication and session management. Verify that the application is not vulnerable to attacks such as session fixation, session sidejacking, or session hijacking.

- **Denial of Service (DoS) Attacks:**
  Check if the application is vulnerable to DoS attacks. Verify that the application has mechanisms in place to prevent or mitigate these attacks.

- **Malware:**
  Check for any instances where the application could be used to distribute malware. This could include insecure file uploads, XSS, or other injection attacks.

- **Third-Party Components:**
  Check if the application is using any third-party components and if so, ensure that they are secure. Verify that the application is not using outdated or vulnerable components.

Here are some of the technical points to consider when doing a pentest security audit of a decentralized application (dApp)-

- **Front-end and Back-end Code Audit:**
  Review the front-end and back-end code of the DAPP for any security vulnerabilities. This includes the code that interacts with the smart contracts, as well as the code that handles user input and data storage.

- **API Security Audit:**
  Review the APIs that are used by the DAPP to ensure that they are properly secured. This includes reviewing the API endpoints, authentication methods, and data encryption.

- **Infrastructure Security Audit:**
  Review the underlying infrastructure that the DAPP is hosted on, such as the blockchain network, cloud servers, and databases. This includes reviewing the security of the infrastructure, as well as the access controls that are in place.

- **The dApp's Business Logic:**
  Understand the DAPP's business logic in order to identify any potential security vulnerabilities.

- **Testing:**
  Perform a variety of tests on the DAPP to simulate real-world attacks. This includes penetration testing, fuzz testing, and stress testing.

04

Development Process Checklist

Development Process Checklist

Smart contracts are a game-changing technology that enables automated, decentralised transactions. However, because of their immutability and the potential for financial consequences, it is critical to create them securely in order to eliminate vulnerabilities and potential losses. Following a detailed development process checklist is one excellent method to accomplish this. Below, we will look at the checklist and how it might assist developers in creating secure smart contracts.

Documentation and Specifications

- Write clear and concise documentation for the smart contract code, including an overview of the purpose and functionality of the contract, input and output parameters for each function, and possible edge cases or limitations.
- Use a standardized documentation format like NatSpec or Doxygen to improve readability and understandability.
- Include a detailed specification document outlining the smart contract's goals, requirements, and constraints.
- Provide detailed instructions on how to interact with the smart contract, including example input and output parameters and any potential gas costs or fees.

Solidity Best Practices and Readability

- Correct Use of Pragma: Avoid using pragma solidity ^0.9.1 when you're setting the compiler version yourself. This can create ambiguity for verifiers regarding the version of solidity used. This pattern should only be used for library code where you're not the one compiling it. Instead, use pragma solidity 0.9.1.

- Refrain from using unexplained constants in your code. It's more effective to use constant variables that describe the value. For instance, instead of using an arbitrary number like 5000, define it as a constant: public uint256 constant MAXIMUM_TRANSACTION_LIMIT = 5000;

- Require statements should always include an explanatory message. Instead of "require(numTokens < MAXIMUM_TRANSACTION_LIMIT)" to "use require(numTokens < MAXIMUM_TRANSACTION_LIMIT, "Number of tokens exceeds transaction limit")"

- Avoid Left-to-Right Character: The Unicode U+202E character, which changes the order in which text is rendered, should be avoided as it's often used for obfuscation purposes.

- Use Descriptive Variable Names, Function Names, and Comments Avoid ambiguous names like mapping3 or data. Ensure that variable and function names are precise, accurate, and descriptive. For example, instead of naming a variable as 'data', name it as 'transactionData'. Also, flag any inaccurate or outdated comments.

- Unused variables can hinder readability. Although the tools mentioned above will catch this, it's important to avoid them. For example, if a variable 'unusedVar' is declared but not used anywhere in the code, it should be removed.

- Proper Use of Readability Keywords (hours, days, ether, 1_000_000, etc.) In Solidity, 1 days will automatically convert to 86400 seconds, which is much more readable. Similarly, use 1_000_000 over 1000000 for readability. When dealing with Ethereum quantities, use the ether keyword. For example, instead of writing 10000000000000000, write 1 ether.

- Use Inclusive Language in Naming While it's not about being politically correct, it's important to remember that your code is stored indefinitely and can be read by anyone worldwide. Therefore, exercise decency, politeness, and respect in your naming conventions. Avoid using offensive or derogatory terms in your code.

On-Chain vs Off-Chain Computation

- Consider using off-chain computation to minimize gas usage and improve performance.

- Use a suitable off-chain computation mechanism, such as an oracle or external API, to provide real-time data to the smart contract.

- Use on-chain computation only when necessary, such as when executing critical functions that require the security and immutability of the blockchain.

Upgradeability

- Implement upgradeability mechanisms to allow for future updates and improvements to the smart contract.
- Use a proxy or delegate contract to separate the contract logic from the contract data, allowing for upgrades without affecting the data.
- Use a versioning system to track changes to the contract and ensure compatibility with existing deployments.
- Implement a governance mechanism to ensure that upgrades and changes are approved by all relevant parties and comply with the requirements of the smart contract.

Function Composition

- Use function composition to break down complex functionality into smaller, reusable functions.
- Use the `internal` visibility modifier for functions that are intended to be used only within the contract, and `**public**` or `**external**` for functions that can be accessed outside the contract.
- Use `**view**` or `**pure**` functions for read-only functionality that does not modify the state of the contract, to minimize gas usage.
- Ensure that composed functions are properly validated and tested to avoid potential vulnerabilities.

Inheritance

- Use inheritance to simplify and modularize contract code.
- Use the `is` keyword to indicate that a contract inherits from another contract.
- Implement abstract contracts to provide a template for child contracts to follow.
- Avoid deep inheritance chains, which can make the codebase difficult to manage and increase the risk of bugs or vulnerabilities.
- Ensure the child contract's implementation follows the requirements and constraints specified in the parent contract.

Events

- Use events to record contract activity and allow for off-chain monitoring and analysis.
- Use the `emit` keyword to trigger an event in Solidity.
- Define the event parameters and their types in the contract to provide a clear and standardized event format.
- Use events to provide feedback to users on the status of contract transactions or state changes.
- Ensure that events are properly validated and tested to avoid potential vulnerabilities.

Avoid Known Pitfalls

- Use dependencies like external libraries or APIs to simplify and modularize contract code.
- Use the latest version of the Solidity programming language and keep up to date with any security-related updates or bug fixes.
- Ensure that external contracts interacting with your business are compatible with each other, including deflation/inflation tokens and reentrant tokens like ERC-777, ERC-677, and ERC-721. Refer to the Reentrancy Vulnerability Case for guidance.
- Consider the risk of reentrancy in external function calls.
- Avoid excessive loops when reading or assigning contract storage variables.
- Avoid concentrating authority, especially the ability to modify critical contract parameters. Instead, use governance mechanisms, timelock contracts, or multi-signature contracts to manage authority.
- Maintain linear inheritance relationships between contracts and ensure that inherited contracts are necessary for business.
- Avoid using on-chain block data as a seed source for generating random numbers.
- Fully consider the possibility of rollback attacks when acquiring and using random numbers.
- Use Chainlink's VRF to obtain reliable random numbers. Refer to the Chainlink VRF for more information.
- Avoid calculating LP Token price directly from the token quantity of third-party contracts. See How to get the price of LP correctly for guidance.

- Use at least three price sources when obtaining prices through third-party contracts to avoid relying on a single source.
- Record the status of execution using events in key business processes for data analysis purposes during the project's runtime.
- Have an emergency switch in place for suspending global and core business activities to prevent further losses in case of a black swan event.
- Ensure that dependencies are properly validated and secure before incorporating them into the contract code.
- Use standardized formats like ABI or JSON to ensure compatibility and interoperability with dependencies.
- Regularly update and maintain dependencies to ensure compatibility with Solidity's latest version and address potential vulnerabilities.

Newsletter

Hashing Bits is a Web3.0 Cybersecurity focused weekly newsletter.
We share the latest hacks, research and tools, as well as career opportunities.
Subscribe now and don’t fall behind.

https://quillaudits.substack.com/ Subscribe

Gas Optimization

- Use `uint256` instead of boolean and small units: A boolean is internally represented as a `uint256` with 255 bits masked out. The extra masking operations incur additional gas costs. It is generally more gas-efficient to use `uint256` for non-storage variables unless there is a specific need to reduce their size.

- Prefer bitshifting over division or multiplication by two: Bit shifting operations (`<<` and `>>`) cost three gas, whereas multiplication and division operations cost five gas. If overflow is not a concern, using bitshifts is a more gas-efficient option.

- Utilize bitmaps for contracts with many boolean variables: Under the hood, a boolean is actually represented as a `uint8`. This means a `uint256` can hold up to 32 boolean variables. If you have more than 32 boolean variables, it is more efficient to store them as individual bits within a `uint256` variable.

- Optimize function names for gas-sensitive functions: Functions with leading zeros in their names can save gas in two ways. First, they are sorted to the top of the bytecode, resulting in fewer checks to determine if that function is called. Second, leading zeros save gas as part of the transaction data. However, this approach can lead to unconventional function names and should only be used when gas savings are significant.

- Pack function arguments for gas-sensitive functions: If you need to represent two 128-bit numbers, consider combining them into a single 256-bit number and splitting them into two numbers within the contract. This technique can save gas. However, it may reduce code readability and should only be used for gas-sensitive functions.

- Avoid using `require(x >= 0, "...")` for units: Unsigned integers cannot be negative, so checking for `x >= 0` is redundant and wastes gas.

- Split `require` statements instead of using boolean operators: Using separate `require` statements for individual conditions is more efficient than combining them with boolean operators.

- Compare boolean literals directly: When `condition` is a boolean variable, using `if (condition)` is more gas-efficient than `if (condition == true)`.

- Prefer `x = x + y` over `x += y`: Although mathematically identical, the compiler treats these operations differently, and `x = x + y` is generally more efficient in terms of gas usage.

- Prefer `++x` over `x++`: If a variable is incremented in isolation, prefix incrementing (`++x`) is preferred over postfix incrementing (`x++`) due to compiler optimizations.

- Use `unchecked { ++x }` when using a loop: If a loop uses `uint256` (which is recommended), and overflow is not a concern, it is safe to increment the loop variable using `unchecked` to remove overflow checks and save gas.

- Avoid initializing storage variables to default values: Storage variables are initialized to their default values (zero, false, etc.) automatically, so explicitly initializing them wastes gas.

- Swap variables in a single line: To swap variables without using an explicit temporary variable, you can use the following syntax:x, y) = (y, x);

- Use `>` or `<` instead of `≤` or `≥` when possible: Solidity represents `≥` as `not <`, which incurs an extra `not` operation and costs three gas. Using `>` or `<` directly is more gas-efficient.

- Use `abi.encodePacked` over `abi.encode` for fixed-length arguments: The `encode` function pads the inputs, which incurs additional gas costs. For fixed-length inputs, using `abi.encodePacked` is more gas-efficient.

- Admin functions can be payable: If a function is meant for administrative purposes and can receive Ether, making it payable can save gas compared to explicitly checking `msg.value` and reverting. However, for functions accessible to the general public, it may still be desirable to make them non-payable to prevent unexpected state transitions.

- Declare the return variable in the function signature: To save a small amount of gas, declare the return variable directly in the function signature instead of using an explicit variable assignment inside the function body.

Testing and Verification

- Write comprehensive unit tests for each function in the contract.

- Use a testing framework, such as Truffle or Remix, to automate testing and provide standardized test results.

- Write integration tests to ensure that the contract works with other components of the system.

- Verify the contract code on a blockchain explorer, such as Etherscan or BscScan, to ensure that it is properly deployed and functioning as intended.

- Consider QuillAudits audits to identify potential vulnerabilities and improve the security of the contract.

Web Front-end Security Configuration Requirements

- Ensure that the entire website uses HTTPS for HTTP communication.
- Configure HSTS to prevent man-in-the-middle attacks, such as DNS hijacking and BGP hijacking. See HSTS Configuration Introduction for more details.
- Configure X-FRAME-OPTIONS to prevent Clickjacking attacks. Refer to X-FRAME-OPTIONS Configuration Introduction for more information.
- Configure X-Content-Type-Options to prevent browser sniffing behavior that may cause security risks. See X-Content-Type-Options Configuration Introduction for details.
- Configure CSP policies to prevent cross-site scripting (XSS) attacks. See CSP Configuration Introduction for more information.
- Configure cookies related to permissions and user credentials with HttpOnly, Secure, Expires, and SameSite flags. Refer to Cookie Configuration Introduction for more details.
- Strictly separate sub-domains of different businesses to avoid XSS problems in one sub-domain affecting others.
- Restrict referenced third-party resources using the integrity attribute to avoid compromising the project's site in case the third party gets hacked. Refer to SRI Configuration Introduction for more information.
- Properly configure CORS to allow only the specified origin domain, protocol, and port to access the project's resources. See CORS Configuration Introduction for details.

- Ensure that the addEventListener/postMessage implemented in the business checks the message's origin and target. See postMessage Configuration Introduction for more information.

Server Environment Security Configuration Requirements

- Select a reputable cloud server provider, such as AWS or Google Cloud.
- Use a strong password and enable MFA for the cloud platform management account used by the project.
- Strengthen the server's security before deploying the project code, including installing HIDS, using SSH Key for login, setting up SSH login alerts, and implementing SSH login Google-auth.
- Use professional software monitoring services, such as APM and Zabbix, to ensure server availability.
- Regularly test the security of the project using professionals, such as QuillAudits.

Quill RedTeam

Be a part of our RedTeam

Learnt smart contract auditing with our FREE Smart Contract Auditing Course and practised your skills with our QuillCTFs?

Test your skills hacking live smart contracts and fetch bounties with the QuillAcademy RedTeam.

$100K+

In Bounties Available

10+

Bounty Platforms

$900M+

Amount Lost to Hacks in 2023

Let’s Break Some Code

05

Incidence Response

Detection and Monitoring

- Implement event logging within your DApp's smart contracts and backend services. Use tools like The Graph or custom event monitoring solutions to track various user activities and transactions.
- Implement strong access controls for your DApp, both on-chain and off-chain. Restrict access to sensitive functions and data to authorized users or entities.
- Monitor gas usage for dApp transactions. Sudden spikes in gas consumption may indicate malicious activity or inefficient contract code.
- If your dApp relies on oracles for external data, ensure that you use reputable oracles and implement checks to verify data authenticity.
- Set up automated alerts and notifications for specific events or conditions that may indicate security incidents, such as large fund transfers or failed transactions.
- Encourage dApp users to report suspicious activities or incidents they encounter while using the application. Establish clear channels for users to submit incident reports, such as through a support portal or contact email.
- Utilize blockchain analytics tools that offer advanced monitoring and analysis of blockchain data. These tools can help you identify irregularities, track transactions, and detect patterns associated with incidents.

Implement Tools to Monitor Smart Contract Activity For

- Large token transfers above a certain threshold.
- Transfers to unknown or blacklisted addresses.
- Unauthorized contract upgrades.
- Unexpected state changes in variables.
- Abnormal transaction volume or gas usage.

Implement Bug Bounty and Responsible Disclosure

- Establish a Bug Bounty Program and Responsible Disclosure Process for security researchers.
- Choose a bug bounty platform or service provider to host your program. Popular choices include QuillRedTeam, Immunefi, HackerOne, Bugcrowd, and Synack.
- Collect feedback from bug bounty(security researchers) to identify areas for improvement.
- Make adjustments based on the feedback received and emerging security threats.

Incident Detection & Reporting Training

- Develop a comprehensive training program covering cybersecurity and hacking indicators.
- Teach employees common signs of a potential hack, such as unusual network activity and phishing attempts.
- Encourage a culture of reporting and establish clear reporting procedures.
- Use real-world examples to illustrate security consequences.
- Regularly update training and conduct cybersecurity drills.

Regular Health Checks of Monitoring Systems

- Schedule routine testing with defined scenarios covering security events and anomalies.
- Mimic real-world conditions for accurate testing.
- Document and analyze test results to identify areas for improvement.
- Adjust detection thresholds based on testing outcomes and threat landscape changes.
- Collaborate with IT and security teams and stay informed about emerging threats.

Pause the Smart Contract Immediately to Prevent Further Damage

- Deploy a "kill switch" that pauses all functionality.
- Freeze token transfers and contract upgrades.
- Revoke approvals for any compromised accounts.

Block Unauthorized Transactions from the Hacker's Addresses

- The first step is to identify the addresses associated with the hacker. This can be done through blockchain transaction analysis, monitoring, or forensics, depending on the specifics of the incident.
- Understand the nature and scope of the security breach. Determine what actions the hacker has taken, which assets they have accessed, and whether any vulnerabilities need to be addressed.
- If the hacker gained access to accounts or wallets, immediately change the credentials (e.g., passwords, private keys) and secure them with strong authentication methods.
- Many blockchain networks and cryptocurrency exchanges have the ability to freeze or lock addresses in certain circumstances. Contact the blockchain network's administrators or the exchange's support team to request this action for the hacker's addresses.

Isolate the Compromised Smart Contract from Other Systems

- Determine which smart contract or contracts have been compromised. It's essential to pinpoint the affected code or contract accurately.
- Develop a strategy for isolating the compromised smart contract. This strategy may vary depending on the nature of the compromise, but generally, it should involve limiting the contract's ability to interact with other systems and the blockchain network.
- If possible, disable or freeze the compromised smart contract. This action prevents further execution of the contract's code and associated transactions.
- Implement access controls or permissions to prevent unauthorised parties from interacting with the compromised contract. This can involve changing the contract's access settings or temporarily revoking privileges.
- In the future, consider using multi-signature wallets for your cryptocurrency holdings. Multi-signature wallets require multiple private keys to authorise transactions, adding an extra layer of security.

Revoke Read/Write Access for Any Compromised Private Keys

- Identify compromised private keys promptly.
- Disable access privileges associated with compromised keys immediately.

- Ensure a secure key revocation process is in place.
- Generate and deploy new private keys as replacements.
- Update access control lists and authentication mechanisms with the new keys.
- Conduct post-incident analysis to understand how the compromise occurred and implement preventive measures.
- Educate employees on key security and incident response protocols.

Eradication and Recovery

- Rollback the smart contract to a secure state before the hack.
- Identify and fix any vulnerabilities in the code that enabled the hack:
  - Logical flaws, unchecked exceptions, reentrancy issues, etc.
- Redeploy the patched smart contract.
- Recover any stolen funds using mechanisms like "pausable token".
- Rotate any compromised private keys.
- Redeploy any related smart contracts that may be impacted.

QuillAudits WAGSI Grants

At QuillAudits, we're dedicated to supporting developers as they continue to BUIDL DeFi protocols, NFT experiences, or any other initiative that advances web3. Each eligible project will be qualified to obtain credits for up to $3,000, redeemable for their project's audit.

Apply Now!

Investigation and Forensics

- Analyze transaction data and contract logs to determine:
  - How much funds were stolen
  - Which accounts or addresses were compromised
  - The sequence of transactions and contract calls
  - When the hack occurred
- Identify the attack vector and root cause of the vulnerability.
- Determine the full impact - funds lost, reputation damage, user trust.

Communication

- Notify users, token holders and community members about the incident.
- Provide regular updates on the response and recovery efforts.
- Be transparent about the root cause, impact and next steps.
- Communicate through all relevant channels:
  - Website notification
  - Social media announcements
  - Email to all users and token holders
  - Posts on community forums

- Manage public perception and reputation through timely, honest communication.

Conduct a "Postmortem" to Identify

- What went wrong and how the hack could have been prevented.
- Any gaps in security processes, monitoring and testing.
- Ways to improve the incident response plan and procedures.
- Implement necessary changes to prevent recurrence:
  - Additional security testing and audits
  - More rigorous code reviews
  - Improved monitoring and alerting systems
  - Incident response training for employees

QuillAudits

$35B Amount Lost to Crypto Hacks

850+ Projects Secured by QuillAudits

800K Lines of Codes Secured by QuillAudits

$30B Amount Protected by QuillAudits

QuillAudits

INDIA

India Blockchain Forum

https://www.quillaudits.com/

audits@quillhash.com

Canada, Dubai, India, Singapore, UK

/quillaudits